Virtual memory management

Clément Ballabriga, Gilles Grimaud, Giuseppe Lipari

Mars 2026

Contents

1 Foreword **4**

2 Virtual Memory for Intel x86 family **4**
2.1 General principles 4
2.2 Functioning 5
2.3 Data structures 6
2.4 Implementation 8
Q1 *Page Directory Entries* 8
Q2 *Page Table Entries* 8
Q3 _ Page directory_ 8
Q4 *Page Table* 9

3 First application **9**
3.1 Addressing structure 9
Q5 compilation addresses 10
Q6 virtual memory kernel 10
Q7 memory map 10
Q8 addressing plan 10
Q9 `setup_mmu()` 11
Q10 implementation 11
3.2 A user land program 11
Q11 space *user* 11
Q12 addressing plan *user* 12
Q13 *handler* of mistakes 12
Q14 program *user* 12
Q15 link edition *user* 12
Q16 activation *user land* 13
3.3 Correction 13
Q17 *Page Fault* v2.0 13
Q18 Investigating the fault 14
3.4 Interposition 14
Q19 *syscall* handler 14
Q20 User-level *syscall* 15
Q21 *sys call* v2.0 15
Q22 Call interposition 15

4 *Stack guard* 15
4.1 stack protection . . . 16
Q23 Sizing the *user space* . . . 16
Q24 Pages and addresses . . . 16
Q25 New `setup_mmu` . . . 16
Q26 Stack overflow . . . 17
Q27 Overflow test . . . 17
Q28 “stack overflow” message . . . 17
Q29 Analysis . . . 17
4.2 Dynamic stack growth . . . 17
Q30 Allocation data structure . . . 18
Q31 Initialization . . . 18
Q32 `void *alloc_page()` . . . 18
Q33 `void free_page(void *)` . . . 19
Q34 Virtual call stack . . . 19
Q35 Implementation . . . 19

5 Heavyweight processes 20
5.1 Memory copy . . . 21
Q36 `void page_copy(...)` . . . 21
Q37 `clone_page_directory(...)` . . . 21
Q38 `free_page_directory(...)` . . . 22
5.2 Activation . . . 22
Q39 *kernel stack* . . . 22
Q40 Execution contexts . . . 22
Q41 Allocator for `struct ctx_s` . . . 23
Q42 A first `ctx_s` . . . 23
5.3 Process switching . . . 24
Q43 `getpid` *syscall* . . . 24
Q44 `switch_to` v1.0 . . . 24
Q45 `yield` *syscall* . . . 24
5.4 Live cloning . . . 24
Q46 `switch_to` v2.0 . . . 24
Q47 `fork()` *syscall* . . . 25
Q48 `fork()` return value . . . 25
5.5 Termination . . . 25
Q49 `exit` *syscall* . . . 25
Q50 user `_entry_point` . . . 25

5.6 Validation 26
Q51 Security 26
5.7 Virtual cloning 26

6 . . . To Be Continued . . . 26

1 Foreword

During the course you have seen that an operating system basically serves three purposes:

1. Managing the hardware to optimize its use;
2. Providing abstractions for applications;
3. Ensuring the security of the computer device to isolate failures or protect software from malicious behaviour.

In this TP you will be interested in configuring a processor device to facilitate memory management (point 1) but also to ensure isolation and protection (point 3). This device, present on many microprocessor families, is called MMU for *Memory Management Unit*.

Using this device you will illustrate two essential paradigms of system programming:

Interposition: which consists in imposing the execution of a system processing as a preamble to the execution of the service actually requested by an application;
Virtualization: which is to give the illusion of the use of material that does not actually exist.

You will experience using an MMU on the x86 family microprocessors in 32-bit protected mode. There is no fundamental difference with 64-bit mode; simply, since the addresses are 64-bit, there are more indirection tables to configure, which needlessly add to the time devoted to the practical work.

Your TD teacher will ask you to fork a “my-kernel” project. Each question on this topic consists of either:

1. Writing a response to the question in a file `rendu.md` at the root of the project, or
2. Adding code to the repository you will have “forked”.

N.B. : Each change must be followed by a commit with a clear and understandable message.

2 Virtual Memory for Intel x86 family

2.1 General principles

The MMU of Intel processors distinguishes between 2 states of operation of the microprocessor. - The *user* status: ~ is the state in which the microprocessor is running an application (even if the user of this application is the *root* of a Unix system for example). - The *kernel* status: ~ is the state in which the microprocessor is: 1. at machine start-up (in practice things are more complicated on intel processors) and; 2. when executing code associated with a hardware or software interruption.

In *kernel* mode, the MMU registers can be reconfigured, in *user* mode, this produces a fault (a kind of software interruption). In fact, in *user* mode, the configuration of the register and the associated

instructions are prohibited and produce faults. Only the *kernel* can manage the device and therefore perform these operations, this is precisely one of its functions…

Intel family microprocessors define (primarily) 3 registers, `CR0`, `CR2`, and `CR3` to configure the MMU in 32-bit protected mode (but also 64-bit).

The `CR0` register (CR stands for *Control Register*) defines a number of bits indicating whether a particular microprocessor mechanism should be implemented or "deactivated".

In this case, bit 31 indicates whether the MMU device is enabled (bit is 1) or disabled (bit is 0). At startup the MMU, which is not yet configured, is not activated either (`CR0` bit 31 is 0). To enable MMU (Intel calls it "Paging" or the name given to bit 31 `PG`), the program (the operating system) needs to move the `PG` bit from 0 to 1. This must be done after the MMU device has been properly initialized, otherwise it will result in undetermined MMU behavior.

The second useful bit in the `CR0` register is bit 16. When this bit is set to 1, it indicates that even in *kernel-mode*, the MMU must generate a fault when the code asks to write a memory address configured as "*Read-Only*", as it would in *user-mode*. If the bit is 0, then the code (*kernel*) can write to a memory address configured *Read-Only* for the *user* code (which will not be able to write anything).

Moreover, when a fault is produced by the MMU, the `CR2` register contains the address that the offending instruction tried to access.

The `CR3` register gives the physical memory address of a table called *Page Table Directory*. The 12 bits of low weight of this address must set to zero. This means that the address of a Table Directory Page is necessarily the start address of a memory page.

2.2 Functioning

In 32-bit protected mode, when enabled, the MMU of Intel processors works as follows.

In its most common configuration, the MMU represents memory as a set of 4096 bytes (4KB) memory pages. The first page (page 0) of physical memory corresponds to bytes with addresses ranging from 0 to 4095, the second page (page 1) of physical memory begins at 4096 and ends at 8191, the third begins at 8192 and ends at 12287, and so on. Hence, all pages start at a multiple address of 4096, that is, at an address whose 12 bits of low weight are set to 0.

The 20 bits of high weight give the page number. In addition, for any memory address corresponding to the address of a byte, the processor just consults the 20 bits of weight of the address to find the correcponding page. The 12 bits of low weight are called *page offset*.

When the microprocessor requests access to an address in memory, that address, which is a 32-bit encoded number (32-bit protected mode), is then called *virtual*. The MMUs device performs the following steps:

Breakdown of the virtual address: The top 10 bits of this virtual address are called `idx1` and the next 10 bits are called `idx2`. Finally, as has already been said, the last 12 bits (of low weight) are called `offset` from page.

Determination of the table page associated with `idx1`: As previously stated, the `CR3` register gives the address of a table, called *Page Table Directory*, of 1024 entries. The MMU looks at this table at the `idx1` index. The code entry, on 32 bits, is different information, including:

1. the indication that this index is valid or not, if it is not valid, the MMU produces a fault;
2. the indication that this index is reserved for kernel mode or not, if it is and the code executed is in user mode, the MMU produces a fault;
3. the 20-bit high-weight address of a table, called *Page Table*, of 1024 entries. The 12 bits of low weight of the start address of this table are necessarily 0, and do not appear in the *Page Table Directory*.

Determination of the physical page associated with idx2: the MMU consults the `idx2` entry of the *pagetable* determined by step 2. The code entry, on 32 bits, contains various information, including:

1. the indication that this index is valid or not, if it is not valid, the MMU produces a fault;
2. the indication that this index is reserved for kernel mode or not, if it is and the code executed is in _user_mode, the MMU produces a fault;
3. the 20-bit high-weight address of a “physical memory page” associated with “(idx1,idx2)”.

Access to the physical address: The MMU then uses the address of the physical page determined by step 3. This address has the particularity of being the address of a page, and therefore, having the 12 bits of low weight set to 0. These 12 bits are replaced by the 12 bits of page offset, produced by step 1. The complete address thus formed is the physical address which is finally accessed (in RAM, or in level 2/3 caches. For reasons of implementation efficiency, the level 1 cache works with virtual addresses).

With this MMU device, it becomes possible to link to each virtual memory page any physical memory page, or to ensure that by attempting to access that page, an application will produce a fault that the operating system can recover in the form of an interruption.

2.3 Data structures

In detail, each entry in a *Page Table Directory* is a 32-bit field organized as follows:

Directory Entry	
bit(s)	**Description**
31-12	bits 31-12 of the physical page associated to this entry. This physical page corresponds to a Page Table.
11-8	0- Not used
7	0- Size of a memory page (0=> 4Ko, 1=> 4Mo)
6	0- Not used.
5	0- Used entries : the MMU set this bit to 1 when it uses it.
4	0- Cache disabled (data is not copied into cache when this bit is set to 1)
3	0- Cache management (0=> write-through, 1=> write back).
2	This entry is authorised in user mode (0) or in kernel mode (1). Access to a kernel entry while in user mode raises an exception (fault)
1	This entry is authorised in reading/writing (1) or in read-only (0). Write acces to an entry marked with 0 raises an exeption (fault)
0	This entry is valid (1) or invalid (0). Access to an invalid entry raises an exception (fault)

Each entry of a *Page Table* is a 32 bit field ordanised as follows

Page Entry	
bit(s)	**Description**
31-12	bits 31-12 of the physical page associated to this entry This physical page is associated to the virtual page
11-9	0- Not used.
8	0- If set to 1, the entry is “global”. In this case, it is the same in all MMU page tables, and it is not invalidated when the table is changed.
7	0- (cache management, not used in this cours)
6	0- « *dirty bit* » (the MMU sets it to 1 when the page is written.
5	0- The MMU sets this bit to 1 when the page is read
4	0- Cache disabled (data is not copied when this bit is set to 1).
3	0- Cache management (0=> write-through, 1=> write back).

Page Entry	
bit(s)	**Description**
2	This entry is authorised in user mode (0) or in kernel mode (1). Access to a kernel entry while in user mode raises an exception (fault)
1	This entry is authorised in reading/writing (1) or in read-only (0). Write acces to an entry marked with 0 raises an exeption (fault)
0	This entry is valid (1) or invalid (0). Access to an invalid entry raises an exception (fault)

2.4 Implementation

You will define the data structures necessary to represent in C a `Page Directory entry` array of 1024 `struct Directory_Entry_s`. Each entry will be 32 bits. For this you can use the *bit field* notation of the C language.

Q1 *Page Directory Entries*

Create a `mmu.c` file in which you declare a `struct Directory_Entry_s`.

Q2 *Page Table Entries*

In the `mmu.c` file, declare a `struct Page_Entry_s` structure corresponding to an entry in a table *Page Table*.

Q3 _ Page directory_

In the `mmu.c` file, declare a global table `page_directory`, composed of 1024 structures `Directory_Entry_s`. You will use the alignment attribute to ensure that the compiler chooses an address aligned to a physical memory page (so aligned to a multiple of 4096, you can define a `PAGE_SIZE` macro).

With `gcc`, the alignment attribute is written as follows:

```
1  int t  __attribute__((aligned (16)));
```

Here the address of the variable `t` is a multiple of 16.

Q4 *Page Table*

In the `mmu.c` file, declare a global table `page_entry`, consisting of 1024 `Page_Entry_s`. You will use the alignment attribute to make sure the compiler selects an address aligned to a physical memory page (thus aligned to a multiple of `PAGE_SIZE`).

3 First application

First, we will study the possibility of using MMU to isolate bugs and malicious behaviors of a «user program» of the operating system. Indeed, when a program does not work properly, it may change the data structures and/or the operating system code itself. In this case, the crash (or malicious behavior) of an application may result in the crash (or compromise) of the operating system itself. To avoid this, the operating system "isolates" from the applications, configuring the MMU so that the application (which will run in _user_mode) cannot modify the memory used by the operating system itself.

To do this, we will use the basic code of `my-kernel` (repository git given by the teacher). The goal is to make it evolve so that it can activate the MMU and execute the code contained in a `user.o` module. This code will be executed in microprocessor *user* mode, with the MMU enabled so that the module `user` cannot access the rest of the memory used by my-kernel.

3.1 Addressing structure

In order to isolate the operating system from the user's program, an addressing plan must be defined. An addressing plan indicates, for each software element, the place (the addresses) it occupies in memory. As you know, an executable program consists of several sections:

The `section .text`: this section of the binary file contains the machine code to execute.

The Data Section and the bss Section: these sections contain the program's global data. The `section .data` contains the initialized data while the `section .bss` contains the uninitialized data. In the data section some compilers distinguish the `section .rodata``, which can only be read, and the ``.data`, which contains data that can be read and edited.

La `section .stack` this section contains the call stack, with local variables.

By default, the compiler selects addresses in accordance with the operating system that will launch the program. However, here we are not compiling software launched by an operating system, but directly "booted" by the "grub". This software loads the binary to run, in physical memory, at a fixed address.

In the Makefile the linker (which selects the memory space of each element of the binary once compiled) is called with an unusual option. This is the "-T" option, which configures the compiler with a "*link script*". This file specifies to the linker which address to put which element of the compiled code.

Q5 compilation addresses

According to the "*link script*" referred to by the "Makefile", to which address is the compiled code stored? Where are the other sections of the compiled program placed?

Q6 virtual memory kernel

When virtual memory is enabled, access to physical memory to read the next instruction to be performed is replaced by access to virtual memory. For this change of address that the memory virtualization suddenly imposes on the microprocessor to happen at best, the simplest is that the virtual address corresponds to the physical address at that time.

In addition, for its display, your program uses the `putc` function that writes to the memory of the video card to display the characters on the screen. You can read in `src/main` :

```
1 /* base address for the video output assumes to be set as character
     oriented by the multiboot */
2 unsigned char *video_memory = (unsigned char *) 0xB8000;
```

Calculate the idx1 and idx2 values of the address at which the '`my-kernel` code is loaded into memory.

Then calculate the idx1 and idx2 values of the video memory address used by the '`putc`' function.

Q7 memory map

You will define a minimum functional virtual memory configuration. It is assumed that you only configure the idx1 entry of the *Page Table Directory* to a *Page Table*, and all the entries of this Page Table (including idx2).

How many memory pages will finally be configured? which corresponds to how many bytes of memory?

Q8 addressing plan

The purpose of this minimal configuration is to ensure that the virtual addresses of the "my-kernel" code physical addresses. In doing so, activating the MMU will not jeopardize execution of the "my-kernel" binary.

Propose, informally, a minimum configuration of MMU tables for physical addresses the binary of my-kernel are the same as the virtual addresses configured by the MMU.

Q9 `setup_mmu()`

Implement a `void setup_mmu()`. This function must:

1. initialize all entries in the `page_directory` table to be invalid (not present);
2. Initialize all entries of `page_table` so that each virtual entry is associated with the same page in physical address (e.g. virtual page 256 will be associated with physical page 256);
3. initialize the `idx1` entry with the address of the `page_directory` table to reference the `page_table`;
4. Initialize registers `CR3` and `CR0` to enable the MMU.

Q10 implementation

Finally, in the `main` of my-kernel function, call your `setup_mmu` function after the others setup are executed. Then check that, with the MMU enabled, your kernel continues to work.

3.2 A user land program

You initialized the MMU to do an identity mapping between the physical addresses of the kernel, and the virtual addresses. This mapping combines the first 4 mega-bytes of physical memory with the first 4 mega-bytes of virtual memory. However, because the *uk* bit of the `Page_Entry` is set to 1, only a code executed in *kernel* mode can run in this virtual address space.

You will now create a second address space, executable by the microprocessor in _use *mode*. This second virtual address space will be used to host a “program” that will not have the privileges of the kernel. It will be in the same CPU right situation as the programs you usually run on your computer. It’s called a “user land” program, that magical world where the crashes of your codes don’t cause the computer to crash. . . .

To do this, the first step is to define a new set of virtual addresses, which will be mapped with the identity function to physical addresses, but with *user* and *read/write* rights.

Q11 space *user*

In the `mmu.c` file, change the `void setup_mmu()`, so that the 4 mega-bytes of memory following those dedicated to the *kernel* are enabled in read and write for a program in user mode. Note that 4 Mbs is a new entry in the ‘`Page_Directory` and 1024 entries in the `Page_Table`. This new table will be called ‘`user_page_table`. Remember to declare it.

Q12 addressing plan *user*

Determine which (virtual) address range you have set up in *user* mode. What is the smallest address available in *user* mode depending on your configuration? And what is the largest accessible address? Enter your answer in the `rendu.md` file.

Q13 *handler* of mistakes

When the user program makes an illegal access to memory, for example by trying to read or write to the kernel memory, it will produce an MMU fault. This raises an *exception*, which is a form of interruption. The interrupt level related to page errors in the MMU is 14. The interrupt vector interface file `idt.h` defines a specific function to record handlers of "software interruptions" (that is, interruptions that originate from software and not hardware) in the IDT. In `include/idt.h`:

```
1  void idt_setup_int_handler(int intno, void (*handler)(int_regs_t *r));
```

Change your `main()` function to set the idt so that in the event of a page fault, the function `page_fault_handler()` is called (similar to the empty_handler). For now the function will only display an error message.

Q14 program *user*

It is now necessary to compile a *user land* program. Since the Makefile already provides a way to produce a bootable binary code, you just need to reuse this compiler string, so that it compiles, in addition to the kernel, a file `src/user.c`, which declares a `void user_main()` function. This function can simply call `puts("Hello world from user land! n")`, then make an infinite loop to maintain the CPU in *user land*.

Q15 link edition *user*

However, the problem is that once compiled, the `user.o` file is linked with the rest of my-kernel code. It will not be placed by the linker at the address you configured in *user* mode!

In order for the code contained in this file to specifically start at the address you calculated at question Q12 you must modify the *link script*.

To do this, simply declare a new section, for example `user_land`, followed by the address to which you want this section to begin, as in the section `.text`, for example.

Within the braces you simply declare the list of files `.o` to put in this section, one file per line. And for each of them, inside parentheses, the list of sections (.text, .data, .rodata, .bss) of the `.o` that you want to place in the *user land* section of the final binary image.

Once the *link script* configuration is complete, verify that the makefile does produce a binary in which the `user_main` function is within the _user_virtual address space.

Q16 activation *user land*

Finally, you have to run the `user_main` function of the `src/user.o` module at the end of your boot sequence (at the end of the `main` of my-kernel). Of course, you could call the `user_main` function, but this function call would not change the operating state of the CPU that is in *kernel* mode and therefore, the MMU would not restrict access to the *user land* memory.

To make this call you must exit the *kernel* mode and for that, the simplest way is to use the assembler statement `iret`, which *returns from an interruption*; so that the return in *user* mode starts the execution of the target function.

Since building a return stack that can be used by the `iret` instruction is tricky (full of technical details), my-kernel's base code provides an implementation in `src/call_user.s`. The file `include/call_user.h` gives the prototype of the `call_user`.

Change your `main` function so that the boot sequence ends with a *user land* call to the `user_main` function, indicating, as a stack, a stack of 64Ko declared in the same file as `user_main`.

Tell us what you observe and why.

3.3 Correction

The goal is now to correct the previous incorrect behavior. At this stage, when a *user land* program triggers a fault, the expected behavior is to display an error message indicating the memory address that caused the fault and the address of the instruction that generated it, and then to halt the system so that the user can read the error message and power off the failed system.

Q17 *Page Fault* v2.0

Propose a new implementation of the *page fault* handler so that it behaves as explained above. The address of the attempted access can be read from the appropriate control register. The address of the instruction that caused the fault can be retrieved from the data structure passed as a parameter to the handler. This structure represents the state of the microprocessor at the time of the interrupt and therefore contains, in particular, the value of the `%eip` register (i.e., the *instruction pointer* on Intel x86 32-bit processors).

Q18 Investigating the fault

State which information you now read when the error occurs, and explain (being explicit about the shell commands you use) how you can use this information to determine the cause of the error. As a reminder, once compiled, the executable binary is written as an ELF file in: `iso/boot/mykernel.bin`.

3.4 Interposition

The issue identified in this section comes from the need to call system functions (sometimes referred to as services) from *user* space.

The simplest solution would be to grant the user access to kernel memory, but that is precisely what we want to prevent: if a user program contains a bug, the operating system must still be able to keep running. As a result, when an interrupt (or a fault) occurs, the user program is suspended and the *kernel* executes a function: the interrupt handler. Microprocessors provide a similar mechanism to transfer control, on demand, from *user* mode to *kernel* mode through a "software interrupt".

To allow user programs to call system functions (i.e., *syscalls*), applications will therefore trigger software interrupts. On x86 processors, a program running in *user* mode can trigger such an interrupt simply by executing the assembly instruction:

```
1       int $87
```

where 87 is the chosen interrupt number. When the microprocessor executes this instruction, it behaves as if a hardware device had raised an interrupt (here, number 87). This therefore triggers the execution, in *kernel* mode, of the interrupt handler attached to interrupt 87.

To "catch" this interrupt in the kernel, you must register an interrupt handler, as done so far, using the function

```
1 void idt_setup_int_handler(int, void (*handler)(int_regs_t *r));
```

Note. In practice, on x86 architectures, other CPU instructions than `int` can be used to implement a *syscall*. Notably, `callf` and `sysenter` can be used (although the latter does not have the same semantics on Intel and AMD processors). We use the `int` instruction here to avoid introducing concepts that are specific to the x86 family (mainly segmentation).

Q19 *syscall* handler

Propose a modification to your main program so that it handles software interrupt 87. This interrupt handler is invoked when a user program requests a system call. For now, the system call we want to support is `puts`. The parameter to this function is the address of the string to print. You must pass this argument to the software interrupt handler. Since the handler receives the register state

of the interrupted program, the simplest approach is to place the parameter in a register. You may (conveniently) assume that the `%edx` register is used by the *user land* program for this purpose.

Q20 User-level *syscall*

You must now modify the program `src/user.c` so that **its** `puts` function triggers software interrupt 87 after writing its argument into the `%edx` register.

Since the kernel already declares a function named `puts`, and you are about to declare another one in the same binary, a naming conflict will occur at compile time. Rename `puts` to `kputs` in `src/main.c`, and declare your new `puts` function in the file `src/user.c`. While doing so, remove the declaration `extern puts(char *s);` from this file.

Q21 *sys call* v2.0

Pour l'instant, l'interruption logicielle 87 ne permet de soliciter qu'un seul service, la fonction `puts`. Cependant nous voulons aussi pouvoir exposer d'autres fonctions. Il s'agit, dans un premier temps, de : `putc` et `clear_screen`. D'autre pourront suivre.

Nous pourrions déclarer un handler pour chaque fonction, mais comme le nombre de fonction système est suspetible d'augmenter avec le futur, alors que le nombre d'interruption est limité à 256, nous allons utiliser un autre moyen. Lors de l'exécution d'une interruption logicielle le registre `%eax` contiendra un nombre 32 bits qui indiquera le numero du service système demandé par le programme utilisateur. Implémentez une nouvelle fonction pour `putc` et `clear_screen` dans `src/user.c`, puis modifiez `sys_call_handler` dans `src/main.c` en conséquence. Testez vos nouvelles fonctions.

Q22 Call interposition

Explain how this implementation of *syscalls* (the one you implemented) protects the kernel from the behavior of a malicious user program.

4 *Stack guard*

In this section, you will set up a virtualization mechanism for the user-accessible memory in order to better manage the execution stack. For now, the stack is simply a global array declared in `src/user.c` and used as the call stack when the user address space is activated (via the call to `call_user`).

This approach is not satisfactory because:

1. if the stack overflows, no error will be raised: local variables will simply overwrite other data placed there by the compiler;
2. the entire stack is reserved *a priori* and may therefore consume more memory than necessary, unnecessarily depriving other programs of this resource.

You will address these two issues in turn.

4.1 stack protection

First, note that the current MMU configuration maps all pages of the second entry of the `page_directory` to physical memory pages. This is not useful here, since your program is much smaller than that. To allocate only the memory that is actually needed, you must know the last address of the `user.o` module. To do so, you can declare, in any section you choose in your linker script, labels such as:

```
1      label = .;
```

By doing this, the linker will consider that there exists a variable **`label`** located at the address where the label is defined. By taking the address of this variable, it becomes possible to determine the address of any part of what the compiler generated.

Q23 Sizing the *user space*

Modify the linker script to declare the variables `_begin_of_user_space` and `_end_of_user_space`, placed respectively at the beginning and at the end of the memory region (and therefore of the section) used by the `user.o` module.

Q24 Pages and addresses

Implement a utility function **`void`** `*page_to_addr(`**`int`** `page)` that returns the address, as a **`void`** `*`, corresponding to the page number given as an argument, as well as its inverse: **`int`** `addr_to_page(`**`void`** `*addr)`.

Q25 New `setup_mmu`

Modify the `setup_mmu` function so that it maps only the physical memory pages that are actually needed by your `user.o` module into the `page_directory` used by the MMU. Check that your kernel still works.

Q26 Stack overflow

To prevent stack overflows, you will simply adjust the MMU configuration so that any attempt to access the last stack page triggers a fault.

This way, when the *user* program pushes too many function calls (for example due to recursion), it will eventually write to this protected page, producing an error that the kernel can intercept, rather than allowing the program to keep running while overwriting its own global variables and/or functions.

Q27 Overflow test

Modify the `user.c` program to test that a stack overflow is indeed caught by the MMU.

Q28 “stack overflow” message

Do a bit of cleanup by moving your page-fault handler from `src/main.c` to `src/mmu.c`. Update `include/mmu.h` accordingly. Indeed, it is becoming increasingly clear that programming this interrupt is closely tied to the chosen MMU configuration.

Then implement (in `mmu.c`) a function `void *get_cr2()` that returns the address that caused the fault.

Finally, modify your interrupt handler so that when a page fault occurs, the message prints **“stack overflow”** instead of **“page fault”** when the error is due to a stack overflow caused by the user program. Of course, in other cases, the generic message remains appropriate.

Q29 Analysis

Explain how it would be possible to detect a stack overflow without relying on this use of the MMU. Then explain what you see as the advantages and disadvantages of using the MMU to prevent execution stack overflows.

4.2 Dynamic stack growth

A major motivation for introducing the MMU in hardware is to make physical memory management easier. Regarding a program’s execution stack, as you can imagine, some programs rely heavily on recursion and therefore need a large call stack. Conversely, more procedural programs may only require a very modest call stack.

In this section, you will allocate memory pages for the execution stack incrementally, as the application needs them. The idea is as follows: the kernel will place the *user* program’s stack on a virtual memory

page that is not present. When the application tries to read from or write to the stack, a page fault will be triggered. The operating system can then allocate a physical page, map it to the running program's stack, and resume execution.

In this way, memory is granted to the application progressively rather than *a priori*. However, in a sense, the operating system is "lying" to the application: it claims to provide memory without actually backing it immediately with physical pages.

To achieve this, you will implement page allocation and page freeing functions. Then you will modify the page-fault handler so that it allocates new pages as needed. If the maximum limit is reached, the error message **"Stack overflow"** will be displayed.

Since allocation will happen on demand, there is no longer any need to declare a `user_stack` variable. The stack will initially be placed at the end of the `user_page_table`. The last entry will not be mapped to a page, and on the first `push` executed by the `src/user.o` module, a fault will trigger the allocation of the first execution-stack page.

Q30 Allocation data structure

To allocate pages, you need to know which physical pages are free and which are in use. Define a data structure that associates an integer with each page (within the first 16 MiB). This integer must be `-1` if the page is reserved; otherwise, it contains the page number of the next free page. In addition, a global variable `first_free_page` must store the page number of the first free physical page. Declare these data structures in the file `src/mmu.c`.

Q31 Initialization

This allocation data structure must be properly initialized in order to work correctly. All pages mapped for the kernel, as well as all pages mapped for the `src/user.o` program, are effectively already allocated. The data structure must therefore be initialized accordingly.

The first free page is the one immediately following the last page of the `src/user.o` module. The next free page is stored in the table at that index, and so on, up to the last free page, which stores `-1` as its "next" page number since there is no free page after it.

Note. Since page `0` may legitimately be free, you should avoid using `0` as a "null page number". The value `-1` is preferable. Update the `void setup_mmu()` function accordingly.

Q32 `void *alloc_page()`

The implementation of `alloc_page()` is now straightforward. You simply return the first free page, after updating the global variable so that the new first free page becomes the "next" free page of the

current one.

In the page table, at the index that you return, the value must be `-1`, meaning: *page allocated*.

Since `alloc_page` returns a page address rather than a page number, you can use your conversion helper `void *addr_for_page(int page)` to convert the returned value.

Implement the function `void *alloc_page()`.

Q33 `void free_page(void *)`

You must now be able to free previously allocated pages. This simply consists in inserting the page to be freed at the head of the free-list, and indicating that the next free page after it is the previous head of the list.

Implement the function `free_page()`.

Q34 Virtual call stack

Compute the values of `idx1` and `idx2` so that the call stack is located on the last virtual page mapped by the `user_page_table`. Assume that the `push` instruction decrements the stack top **before** writing the value. What must the stack address be so that the instruction writes to that page?

Q35 Implementation

Modify the page-fault handler so that it allocates a new page each time an access to the stack fails, and then resumes execution instead of failing. With this behavior, a program without infinite recursion should be able to run normally.

Set a maximum limit of 1 MiB for stack growth. Allow the stack to grow up to this threshold and print **“stack overflow”** when the limit is reached.

Verify that everything works correctly by testing stack growth all the way up to the **“stack overflow”** condition.

/ ! \ Congratulations, you have just implemented your first virtualization mechanism!

---

5 Heavyweight processes

You will now implement the notion of a heavyweight process. A heavyweight process is an execution context associated with its own virtual memory space. Each running program executes within "its" own memory, meaning that it reads from and writes to pages that are private to it and distinct from those of other programs. In addition to its execution stack, each process therefore has its own address space, configured through the `%cr3` register. Switching from one process to another means switching the virtual address space. Each process thus has its own `directory_table`.

The POSIX API you are familiar with defines the function `fork()` to create processes. In this API, each process is identified by a unique number called a `pid`. You are now going to implement this function.

As a reminder, the principle of `fork()` is to duplicate the memory of a process in order to create a second one. The newly created process is therefore an exact copy of the original. The only difference between the two lies in the return value of `fork()`. The original process receives, as the return value of the call, the `pid` of the newly created process, whereas the new process—continuing execution from the same `fork()` return point—receives the value `0`. This difference allows the original and the copy to adopt different behaviors afterward.

If the memory of the initial process is duplicated, there is no need to duplicate the kernel pages themselves. Indeed, regardless of which user program is executed, the operating system remains unique.

To implement process duplication, you must also preserve the execution state of each process. This execution state is stored in the execution stack that has been duplicated. However, you must also know:

1. the address space associated with the process (as given to the `%cr3` register),
2. the top of the stack (defined by the `%esp` and `%ebp` registers), and
3. the `pid` of the currently running process.

Together, these elements constitute an execution context.

Once an execution context has been associated with each process by the kernel, it becomes possible to implement the system call `yield()`, which switches from one process to the next. In doing so, it changes the virtual address space. It is precisely because the address space changes from one process to another that we speak of heavyweight processes. Indeed, reconfiguring virtual memory at each time quantum has a non-negligible impact on performance. Lightweight processes, which share the same virtual memory space, are more efficient. However, they are not isolated from one another, and a bug in one may cause all of them to crash.

Once your first program `src/user.c` is correctly started, so that it is attached to an execution context and scheduled by the `yield` system call, it becomes possible to implement the `fork` function, which creates a copy of this first process and integrates it into the scheduler.

To implement `fork`, you will therefore:

1. copy the virtual memory of a process;
2. handle the activation of the first process;
3. implement context switching between processes;
4. perform a live cloning of a process state;
5. handle process termination by freeing the associated memory.

5.1 Memory copy

To implement cloning, you will write the function:

```
1 struct Page_Directory_s *clone_page_directory(struct Page_Directory_s *
    src_pd);
```

This function allocates and initializes a new `page_directory`. As with the original, all entries are marked as "not present" except for the first two. The first entry is identical to the original, since it corresponds to the kernel code, which is unique. The second entry references a new `page_table` in which each entry is a copy of the source. However, when an entry is marked as present, it must reference a newly allocated page that contains a copy of the original page's contents.

Q36 `void page_copy(...)`

First, you need a function that copies a memory page. This function, with prototype `void page_copy(char *pg_src, char *pg_dst)`, copies the memory page pointed to by `pg_src` into the page pointed to by `pg_dst`.

Since these addresses refer to physical memory, you must temporarily disable virtual memory before performing the copy, and re-enable it afterward. To perform the actual copy, you will implement a simple loop. The `memcpy` function from the `libc` is, of course, not available within your kernel.

Q37 `clone_page_directory(...)`

Implement the function

```
1 struct Page_Directory_s *clone_page_directory(struct Page_Directory_s *
    src_pd);
```

This function allocates and initializes a new `struct Page_Directory_s` as described above, as well as a new `user_page_table` using the `alloc_page()` function. Then, additional pages are allocated for each valid entry in the `user_page_table`. You can then use the `page_copy` function to clone the process memory contents.

Q38 `free_page_directory(...)`

Implement the function

```
1 void free_page_directory(struct Page_Directory_s *src_pd);
```

This function frees all *user* pages allocated in the `page_directory` passed as an argument, as well as the `user_page_table` and the `page_directory` itself. Of course, you must not free the *kernel* pages (associated with entry 0), since they are shared among all processes.

5.2 Activation

By cloning the contents of the user program's virtual memory, you have *de facto* cloned the *user-mode* call stack (the one you dynamically allocated). You have therefore created a new process, a clone of the initial one, but with a stack whose contents can evolve differently in the future without overwriting the original.

However, so far you have not addressed the kernel call stack—the one used to run your `main` function during *boot*. This stack is defined in the compilation linker script and set up in the file `src/0boot.s`. For those who are curious, this stack is also defined as the interrupt-handling stack in the file `src/gdt.c`.

Q39 *kernel stack*

The goal is to create one kernel execution stack per running program. Currently, entry 0 of each process's `page_directory` is always linked to the same initial `page_table`. You will therefore move the kernel stack into the memory associated with entry 1 of the different `page_directory` instances. By doing so, when switching processes, the microprocessor will also switch the memory used for the *kernel* stack.

Modify the linker script to move `_kernel_stack` to the end of the 4th page (index 3) of the `user_page_table`. Move the `src/user.o` module right after these first four pages. Modify the `setup_mmu()` function so that the pages of the relocated kernel stack are accessible only to the kernel. Then verify that everything still works correctly after these changes.

Q40 Execution contexts

To be able to switch between different processes, you must manage a set of contexts. An execution context is, as discussed previously, defined by the top of the call stack (handled by `%esp` and `%ebp` on Intel processors), but also by its own virtual address space (handled through the `%cr3` register).

In the file `include/ctx.h`, provide the implementation of a data structure `struct ctx_s` that will be used to store the information related to a process. This information includes:

- the `pid` (a unique number associated with each process, equal to 1 for the first process);
- the `cr3` value that defines the process address space;
- the `esp` and `ebp` registers that define the resume position of the execution stack;
- and finally `next`, which stores the address of the next process in the list.

The global variable `current_ctx`, declared in `src/ctx.c`, stores the address of the current context.

Q41 Allocator for `struct ctx_s`

When needed, the kernel allocates a new `ctx_s` structure in memory. This structure will be freed when the process whose state it stores terminates.

1. Define a constant `MAX_CTX` in `include/ctx.h` that sets the maximum number of contexts that can exist simultaneously.
2. In `src/ctx.c`, declare a global array `ctxs` of `MAX_CTX` contexts, as well as a global variable `first_free_ctx`.
3. Implement a function `void init_ctx()`. This function initializes `first_free_ctx` with the address of the first entry in the array. The `next` field of the first entry points to the second entry, and so on, up to the last entry in the array, whose `next` field is `NULL`. You thus build a free-list of `struct ctx_s`.
4. Implement a function `struct ctx_s *alloc_ctx()` that returns the address of a free `struct ctx_s`. To do so, it simply returns the value stored in `first_free_ctx` and updates this global variable so that it points to the next (`next`) free context. Of course, if `first_free_ctx` is `NULL`, the function returns `NULL` (there is no `struct ctx_s` available anymore in your kernel).
5. Implement a function `void free_ctx(struct ctx_s *)` that inserts the structure passed as an argument at the head of the free-list of available `struct ctx_s`.

Make sure that `alloc_ctx` returns a structure whose `pid` field is properly initialized (incremented by 1 compared to the last context that was returned).

Q42 A first `ctx_s`

Modify your boot function (`void main()`) so that it initializes context management (using `init_ctx()`). Then allocate a first execution context, assign it to `current_ctx` for the first created process, and launch it (the process in `src/user.c`).

The `next` field of this first active process should point to itself, since it is the only active one for now.

5.3 Process switching

Q43 `getpid` *syscall*

Implement a new system call: `getpid()`. The kernel function `getpid()` returns the `pid` of the current process.

To do so, the *user land* function declares a local variable and places the address of this variable in `%edx`. The system's `sys_call_handler` then retrieves the address of this local variable from the `%edx` register value received as a parameter, and writes the expected result to that address. Finally, the *user land* function reads the value back and returns it to the caller.

Test that this function works correctly.

Q44 `switch_to` v1.0

You will now implement a function `void switch_to(struct ctx_s *ctx)` that saves the current execution context into `current_ctx` before restoring the context `ctx`, which becomes the new current context.

The `switch_to` function performs the context switch by: 1. saving the stack registers (`%esp` and `%ebp`), then 2. setting `ctx` as the new `current_ctx`, and 3. restoring the stack registers of the new `current_ctx`, and most importantly the `%cr3` register, which reconfigures virtual memory for the new execution context.

Implement the function `void switch_to(struct ctx_s *ctx)`.

Q45 `yield` *syscall*

Implement a function `kyield()` that simply transfers execution from the current program to the next one. Since you have built a ring starting from `current_ctx`, the implementation of `yield` is straightforward.

5.4 Live cloning

Q46 `switch_to` v2.0

In its current form, `switch_to` transfers the execution flow to the context passed as an argument. However, when creating a cloned process, you must put the CPU in a situation where it will later be able to resume execution by switching to that clone. To do so, you will extend `switch_to` so that it creates a clone from the current context when `NULL` is passed as its argument.

The clone is simply inserted into the ring right after the current context. Since the clone is an exact copy of the current context, the stack top (registers `%esp` and `%ebp`) can be saved into the current context. However, the virtual-memory configuration register `%cr3` must be the one obtained through cloning (returned by your function `struct Directory_Entry_s *clone_page_directory(struct Directory_Entry_s *original);`) and not the original one.

Also note that when `switch_to` is called with `NULL`, it does not change the current context and therefore does not transfer execution to the clone. The actual "switch" will occur on the next call to `kyield()`, since the new process is inserted *after* the current process in the ring.

Q47 `fork()` *syscall*

Implement a new system call: `void fork()`. This system call invokes the function `kfork()`, which in turn calls `switch_to()` with `NULL` as its argument in order to create a cloned process.

Q48 `fork()` return value

According to the POSIX standard, `fork()` returns `0` in the child process, and the child's `pid` in the parent process. Implement these return values and verify that they are correct from the point of view of the `src/user.c` process.

5.5 Termination

Q49 `exit` *syscall*

Implement the system call `void exit()` which terminates a process. To do so, the call removes the current process from the list of eligible processes by making its predecessor point to its successor. Then it frees the associated context (`struct ctx_s`). Finally, it frees all pages allocated to this process by calling:

```
1 void free_page_directory(struct Page_Directory_s *src_pd);
```

Q50 user `_entry_point`

When the function `void user_main()` returns, the process must terminate by calling the system call `void exit()`. To achieve this, instead of calling the developer-provided function directly, the kernel will call the *user*-space function `void _entry_point()`. This function calls `void user_main()`, and if `user_main` returns, `_entry_point` calls `exit`.

Implement the function `void _entry_point()` and make sure it is the one invoked by the kernel.

5.6 Validation

Q51 Security

Explain why it is “physically” (i.e., at the hardware level) impossible for the child process to access a secret value stored in one of the parent’s variables **after** the child has been created. Why is it possible **before** the fork?

Provide your explanation in `rendu.md`, and propose a program `src/user.c` that illustrates your argument.

5.7 Virtual cloning

6 ... To Be Continued ...